

CS336 Assignment 5 Alignment 总结报告

2026 年 6 月 5 日

1 总体成果

本次完成的是 Assignment 5 的 Alignment / Reasoning RL 本地实现部分。核心工作是把 `tests/adapters.py` 中的接口接到真实实现，新增 `cs336_alignment/alignment.py`，覆盖 GRPO 训练所需的 tokenization、response log-prob、reward、advantage normalization、policy-gradient loss、microbatch aggregation、single train step，以及 supplement 中的 packed SFT dataset、MMLU/GSM8K 解析和 DPO loss。

最终本地验证结果如下：

验证命令	结果	说明
<code>python -m pytest -q</code>	26 passed	本地 Python 环境全量测试
<code>UV_CACHE_DIR=.uv-cache uv run pytest -q</code>	26 passed	课程 uv/Python 3.12 环境全量测试
<code>UV_CACHE_DIR=.uv-cache bash test_and_make_submission.sh</code>	19 passed	提交脚本要求的 GRPO 测试

同时生成了 `test_results.xml` 和 `code.zip`。打包脚本已修正为不把 `.uv-cache` 和 `.venv` 放入提交包。由于当前机器没有可用的大模型训练资源，也没有课程共享 Modal/SUNET 环境，本次没有实际运行 OLMo/Llama 的 B200 级训练实验；报告中的“实验结果”主要来自课程单元测试、snapshot 数值校验和打包脚本验证。

2 阶段一：阅读与接口定位

任务是什么：阅读 Assignment 5 讲义、README、tests 和 adapter，确定需要实现的接口。README 指明需要完成 `tests/adapters.py`，必需测试为 GRPO；目录中还包含 supplement 的 data、metrics 和 DPO 测试。

用到的大模型知识：本阶段主要梳理 alignment pipeline：prompting baseline、GRPO 作为 reasoning RL 算法、SFT instruction tuning、DPO preference optimization，以及评测解析。

实验结果：初始运行 `python -m pytest -q` 得到 26 个失败，全部来自 `NotImplementedError`，说明环境和 fixture 可用，失败原因是接口未实现。

3 阶段二：Prompt/Output Tokenization 与 Log Prob

任务是什么：实现 prompt 和 output 分开 tokenize 后拼接，并构造 causal LM 训练所需的 `input_ids`、右移一位的 `labels` 和只覆盖 response label 的 `response_mask`。随后根据模型 logits 计算每个 label token 的条件 log probability 和可选 entropy。

用到的大模型知识： causal LM 的训练目标是 $\log p(x_t | x_{<t})$ 。因此模型输入是序列前缀，label 是右移 token；RL 只优化模型生成的 response，不应把 prompt token 纳入 policy-gradient loss。

关键代码：

```
input_ids[row, :length] = torch.tensor(full_ids[:max_len])
labels[row, :len(row_labels)] = torch.tensor(full_ids[1:max_len + 1])
response_mask[row, :len(row_labels)] = torch.tensor(
    [(label_position + 1) >= prompt_len
     for label_position in range(len(row_labels))]
)

log_probs_all = F.log_softmax(logits, dim=-1)
log_probs = torch.gather(log_probs_all, dim=-1,
                        index=labels.unsqueeze(-1)).squeeze(-1)
```

实验结果： test_tokenize_prompt_and_output 和 test_get_response_log_probs 通过。第一次实现时短序列 padding 对齐错误，修正为 full_ids[:max_len] 与 full_ids[1:max_len+1] 后通过 snapshot。

4 阶段三：Rollout Reward 与 Group Advantage

任务是什么： 对 rollout response 调用 reward function，返回 raw reward 和 metadata；再按 group size 做 GRPO/Dr.GRPO/MaxRL 风格的 advantage normalization。

用到的大模型知识： GRPO 使用同一个 prompt 下多个 rollout 的组内相对奖励作为 advantage，通过组均值 baseline 降低方差；不同算法变体改变 baseline、normalizer 或 loss 聚合方式。

关键代码：

```
grouped = flat_rewards.reshape(-1, group_size)
centered = grouped - grouped.mean(dim=1, keepdim=True)
advantages = centered / (grouped.std(dim=1, keepdim=True) + advantage_eps)
```

实验结果： compute_rollout_rewards、compute_group_normalized_rewards_grpo、compute_group_normalized_rewards_maxrl 全部通过。

5 阶段四：Policy Gradient Loss 与 GRPO Train Step

任务是什么： 实现 on-policy loss、off-policy importance reweighting、GRPO/PPO token-level clipping、GSPO sequence-level clipping，并实现一个完整 single-batch GRPO 训练步。

用到的大模型知识： policy gradient 最大化 $A_t \log \pi_\theta(a_t | s_t)$ ，实现中以 negative objective 作为 loss。off-policy 场景需要 importance ratio $\exp(\log \pi_\theta - \log \pi_{\text{old}})$ ；clip objective 用于控制策略更新幅度。GSPO 把 token ratio 聚合为 sequence-level geometric ratio。

关键代码：

```
ratio = torch.exp(policy_log_probs - old_log_probs)
clipped_ratio = torch.clamp(ratio, 1.0 - cliprange, 1.0 + cliprange)
objective = torch.minimum(advantages * ratio, advantages * clipped_ratio)
loss = -objective
```

```
sequence_log_ratio = (log_ratio * response_mask).sum(dim=1, keepdim=True) / denom
sequence_ratio = torch.exp(sequence_log_ratio)
```

实验结果: on-policy、noclip off-policy、GRPO clip、GSPO clip、sequence normalization、constant normalization, 以及标准/变体 train step 测试均通过。全量 GRPO 测试在提交脚本中为 19 passed。

6 阶段五: SFT Packed Dataset 与 Batching

任务是什么: 实现 instruction tuning 的 packed dataset, 把 Alpaca 格式样本 tokenize 后连续拼接, 并切成固定长度 chunk; 同时实现 dataloader batching。

用到的大模型知识: SFT 仍是 causal LM next-token prediction。packing 可以减少 padding 浪费, 提高 token 利用率; 样本之间需要 EOS 边界, 避免模型把两个独立 instruction-response 样本误看作连续语义。

关键代码:

```
text = prompt_template.format(
    instruction=row["prompt"], response=row["response"]
).rstrip()
token_ids.extend(tokenizer.encode(text, add_special_tokens=True))
token_ids.append(tokenizer.eos_token_id)
```

实验结果: test_packed_sft_dataset 与 test_iterate_batches 通过。第一次实现漏掉 EOS, snapshot 中样本边界不匹配; 追加 EOS 后与预期 tokenized sample 对齐。

7 阶段六: MMLU/GSM8K 解析

任务是什么: 从模型输出中解析 MMLU 的 A/B/C/D 选项, 以及 GSM8K 的最终数字答案。

用到的大模型知识: 自动评测需要把自由文本 generation 映射为结构化答案。解析函数应尽量稳健: MMLU 优先匹配 “answer is”、“answer:”、“option” 等模式; GSM8K 取输出中最后一个数字作为答案。

关键代码:

```
match = re.search(r"\b(?:answer\s+is|answer:|option)\s*([ABCD])\b",
    model_output, flags=re.IGNORECASE)
matches = re.findall(r"[-+]?(\d[\d,]*)?(?:\.\d+)?", model_output)
```

实验结果: test_parse_mmlu_response、test_parse_mmlu_response_unknown、test_parse_gsm8k_response 和 test_parse_gsm8k_response_unknown 全部通过。

8 阶段七: DPO Loss

任务是什么: 给定 policy model、reference model、prompt、chosen response 和 rejected response, 计算单样本 DPO loss。

用到的大模型知识: DPO 直接优化 preference pair, 不显式训练 reward model。其核心是比较 policy 相对 reference 的 chosen/rejected log-likelihood gap:

$$\mathcal{L}_{DPO} = -\log \sigma(\beta[(\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)) - (\log \pi_{ref}(y_w|x) - \log \pi_{ref}(y_l|x))])$$

关键代码：

```
dpo_prompt = prompt_template.format(instruction=prompt, response="")
chosen_response = response_chosen.rstrip() + tokenizer.eos_token
rejected_response = response_rejected.rstrip() + tokenizer.eos_token
preference_logit = beta * (
    (chosen_logp - rejected_logp) - (chosen_ref_logp - rejected_ref_logp)
)
return -F.logsigmoid(preference_logit)
```

实验结果： `test_per_instance_dpo_loss` 通过。调试中发现裸 prompt 得到 loss 1.1436，不符合期望；使用完整 Alpaca 模板并把 EOS 计入 response 后得到 0.91038，与单测期望 0.9104 对齐。

9 失败尝试记录

1. 基线运行 `python -m pytest -q`: 26 failed, 全部是 adapter 未实现。
2. 首次实现后运行全量测试: 23 passed, 3 failed。失败点分别是 tokenization padding/shift、packed SFT EOS 边界、DPO prompt 模板。
3. 第一次运行 `bash test_and_make_submission.sh`: 脚本继续打包，但 `uv run` 因默认缓存目录 `/home/ldz/.cache/uv` 只读而失败。
4. 使用本地 `UV_CACHE_DIR=.uv-cache` 后，`uv` 又因网络沙箱无法下载 `flash-attn wheel` 元数据而失败。联网授权后通过。
5. 为运行 `uv` 临时生成的 `.venv` 和 `.uv-cache` 占用大量空间；验证和打包完成后已删除。

10 最终状态

最终代码层面完成了 assignment5 本地可测的全部接口，包含 required GRPO 和 supplement 中的 data、metrics、DPO。最终产物包括：

- `cs336_alignment/alignment.py`: 核心实现。
- `tests/adapters.py`: adapter 接口接入实现。
- `assignment5_work_log.md`: 失败尝试与验证记录。
- `assignment5_report.tex`: 本总结报告。
- `test_results.xml`: 提交脚本生成的测试报告。
- `code.zip`: 最终打包文件。

受限于本机没有可用 B200/Modal/SUNET 课程资源，本次没有执行讲义中需要大模型 GPU 训练的 OLMo-2/Llama-3.1 实验、AlpacaEval 评测和安全红队实验；这些属于环境资源缺口，而不是本地接口实现缺口。